UNIVERSIDAD MANUELA BELTRÁN · INGENIERÍA DE SISTEMAS · SISTEMAS TRANSACCIONALES

Hotel Reservas

Sistema transaccional móvil (APK) para la gestión de reservas de habitaciones

Gustavo Adolfo Cardona Bayona · Jonathan Soler Guevara · Hugo Andrés Forero Angel

Docente: Jamilton Fernando Benvides · Segunda auditoría presencial · noviembre de 2026

El problema y lo que prometimos resolver

Problemática (Fase I)

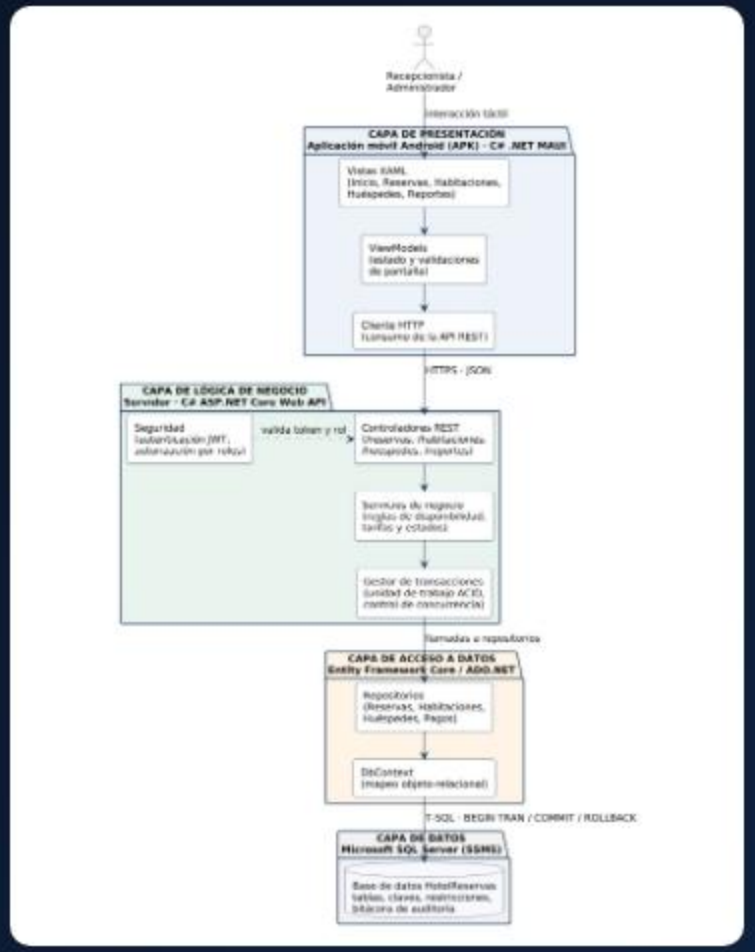
- La disponibilidad se lleva en cuadernos, hojas de cálculo y mensajes: cada recepcionista ve una versión distinta.
- Dos reservas pueden caer sobre la misma habitación y las mismas fechas: **sobreventa**.
- El comprobante se transcribe a mano y el historial no se puede reconstruir.

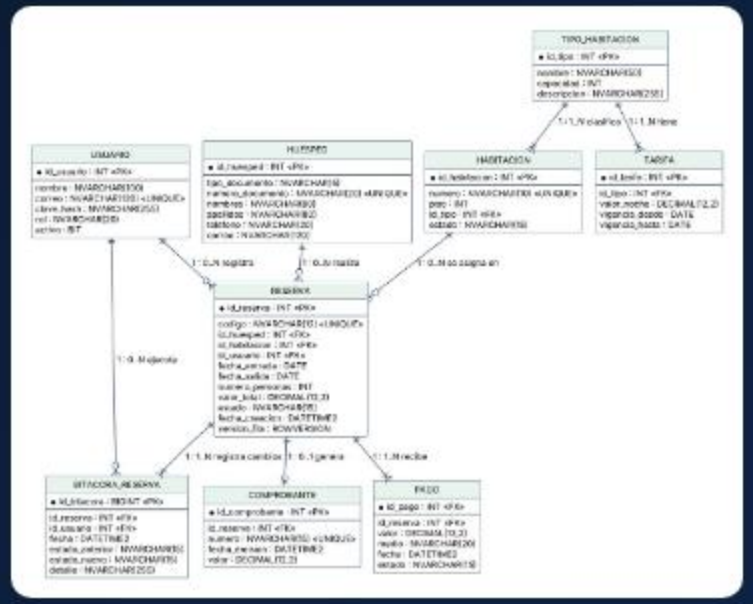
Objetivo central

Un registro único y concurrente: **confirmar reserva** es una transacción ACID que verifica disponibilidad bajo bloqueo y registra reserva, pago, comprobante y bitácora, o no deja nada a medias.

25	4	21	5
habitaciones	tipos por personas	requerimientos funcionales	RNF con métrica

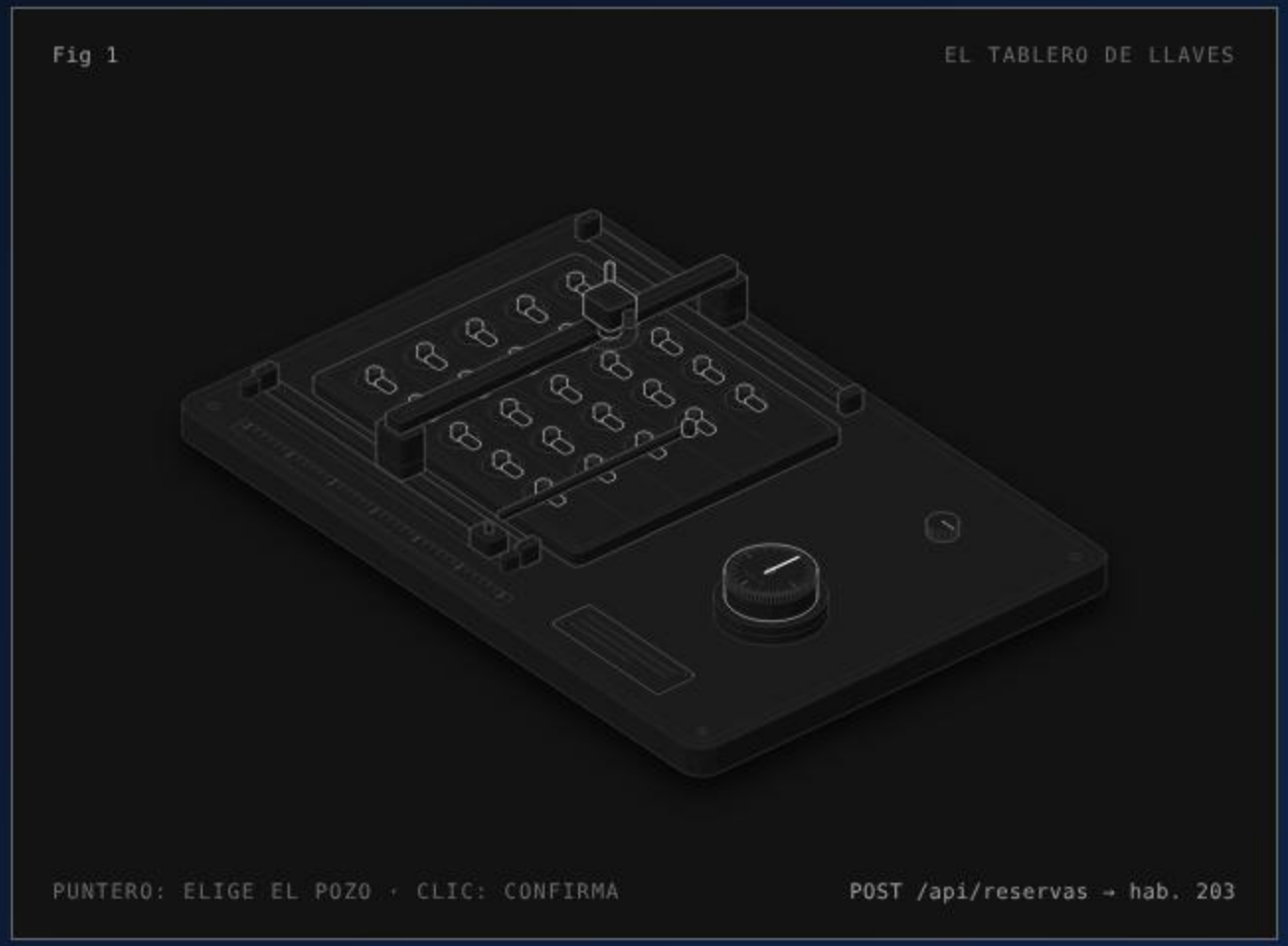
Arquitectura en capas y modelo de datos





- **Presentación:** app Android (Kotlin + WebView). **Lógica:** API REST ASP.NET Core 8 con JWT. **Datos:** Entity Framework Core y SQL Server.
- La app nunca toca la base de datos: todo pasa por la API; a la segunda sesión que pide la última habitación SQL Server le responde con **409**.
- Nueve tablas (USUARIO, HUESPED, TIPO_HABITACION, HABITACION, TARIFA, RESERVA, PAGO, COMPROBANTE, BITACORA_RESERVA) más CONSULTA_ASISTENTE.
- RESERVA: estados PENDIENTE a CANCELADA, valor_total = subtotal + impuesto (CHECK), ROWVERSION y riesgo_cancelacion.
- Scripts 01 a 08; contraseñas PBKDF2-SHA256; token JWT de 480 minutos con el rol.

Cómo funciona confirmar reserva, como una máquina



- **La bandeja** es la tabla RESERVA: un pozo por habitación, una llave por noche disponible.
- **El puente** es la API: recibe POST /api/reservas con el token y lleva la transacción hasta la habitación pedida.
- **El cerrojo** es SET TRANSACTION ISOLATION LEVEL SERIALIZABLE con UPDLOCK y HOLDLOCK: mientras baja, nadie más toca esa habitación.
- **La pinza** toma la llave: INSERT en RESERVA, PAGO, COMPROBANTE y BITACORA_RESERVA en una sola unidad de trabajo.
- **El tambor** es la bitácora: avanza un paso por cada COMMIT.
- **La segunda mano** es otra sesión: choca con el cerrojo, hace ROLLBACK y recibe HTTP 409 Conflict. Ninguna fila queda a medias.

`database/02_transaccion_confirmar_reserva.sql` · `ServicioReservas.ConfirmarAsync` · `ReservasController.Confirmar`

Requerimientos y perfiles de usuario

Tres perfiles

- **Administrador:** tarifas, habitaciones, usuarios y clientes; elimina reservas y recepcionistas (la cuenta admin está protegida).
- **Recepcionista:** disponibilidad, reservas, pagos, check-in, check-out, cancelación, historial de su actividad.
- **Cliente:** reserva, paga, consulta sus reservas y recibe la confirmación por WhatsApp o correo.

Requerimientos no funcionales con métrica

- **Seguridad:** 100 % de solicitudes sin token válido rechazadas (401/403); 0 contraseñas en claro.
- **Integridad:** 0 reservas solapadas tras la prueba de concurrencia; 0 filas parciales tras un rechazo.
- **Rendimiento:** disponibilidad en menos de 2 s y confirmación en menos de 3 s (p95, 10 usuarios).
- **Usabilidad:** una reserva en máximo cinco pantallas y tres minutos.
- **Trazabilidad:** cada cambio de estado con usuario y fecha en la bitácora.

La aplicación, pantalla por pantalla



Inicio de sesión: tres perfiles y modo conectado

- Sencilla (1 persona, 120.000), Doble (2, 180.000), Triple (3, 220.000) y Cuádruple (4, 260.000); baño privado en todas.
- Al tocar un tipo aparecen de inmediato sus habitaciones disponibles; segunda opción, fotos y detalles.
- Confirmación con subtotal, IVA 19 % (exento para extranjero no residente) y total con IVA.
- Apartar sin pago: el huésped tiene 24 horas; vencido el plazo la reserva se cancela sola.
- Comprobante por WhatsApp o correo; riesgo de cancelación de la red neuronal.

Modo conectado: lo que entra por el teléfono queda en SQL Server



Disponibilidad calculada por la API sobre SQL Server

- Casilla **Conectar con la API** en el inicio de sesión: login, disponibilidad, huéspedes, reservas, historial, check-in y cancelación contra HotelReservas.Api con token JWT.
- **201 Created**: código RSV y comprobante CMP generados por sp_ConfirmarReserva; se comprueban en SSMS.
- **Historial y bitácora** leídos con GET /api/reservas/{codigo}.
- **409 Conflict** al repetir habitación y fechas: la transacción hizo ROLLBACK.
- El botón Ver servidor muestra cada llamada HTTP con su código.



Servicio de inteligencia artificial: dos redes neuronales





RF19 · Riesgo de cancelación

- Perceptrón 8-8-1: anticipación, noches, personas, tipo, medio de pago, huésped frecuente, fin de semana.
- Exactitud 88,8 %, precisión 88,1 %, sensibilidad 90,2 %, F1 89,2 % (validación, 80 reservas).

RF21 · Ocupación a siete días

- Red de regresión 5-6-1 evaluada con la ficha técnica: MAE 1,00 · MSE 1,37 · RMSE 1,17 · R^2 0,838.
- Servicio TensorFlow/Keras + FastAPI con diez pruebas automáticas (10/10) y puente /api/prediccion en la API.

Validación, primera auditoría (3,9) y plan de mejora

3,9	3,0	3,0	5,0
calificación	coherencia	evidencia del prototipo	material

Pruebas ejecutadas

- 401 con credenciales incorrectas; fechas invertidas y exceso de personas validados antes de llamar; documento repetido → 409.
- Misma habitación y fechas → 409 y ROLLBACK sin filas nuevas; concurrencia en dos ventanas de SSMS (script 04, error 50001).
- Diez pruebas automáticas del servicio inteligente; pruebas con Playwright del prototipo.

Lo que pidió el docente y cómo se atendió

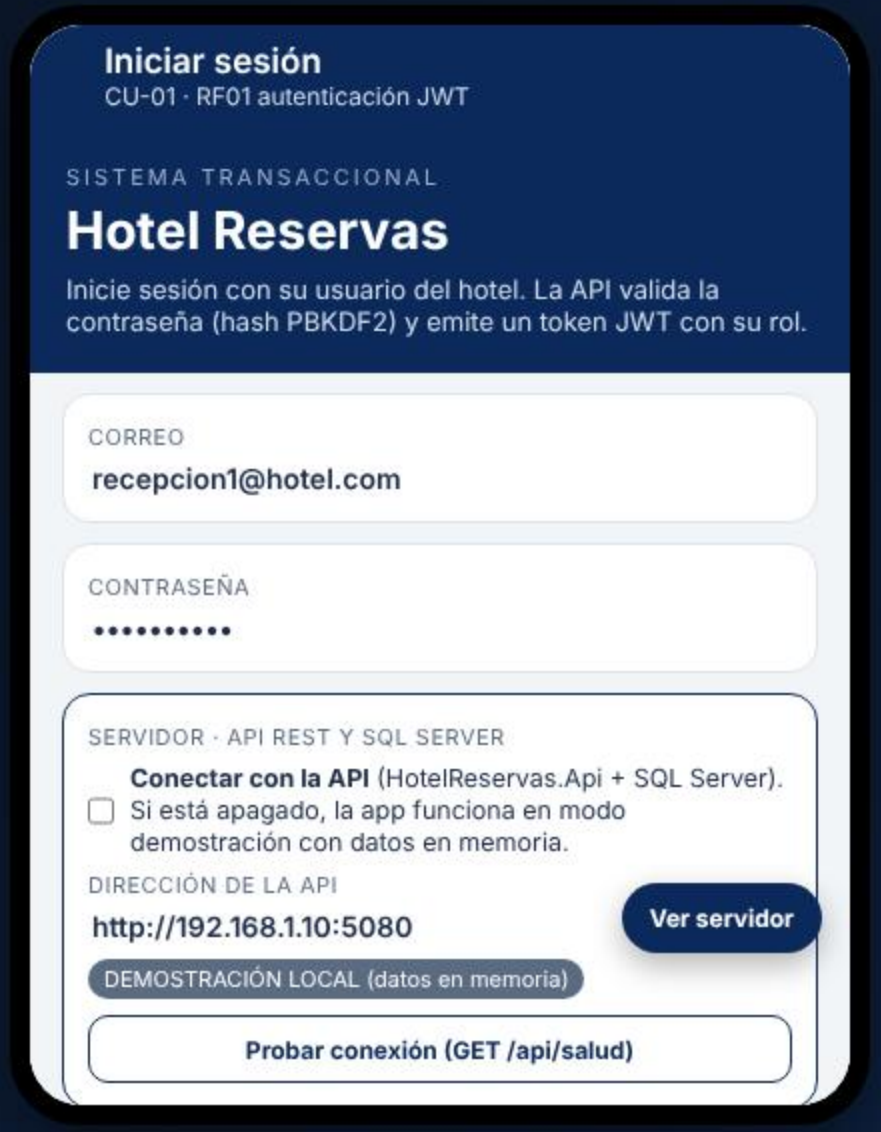
- Sección 16 del documento: matriz de pruebas, evidencias y tabla de coherencia (criterios 2 y 3); estados FINALIZADA unificados y perfil Cliente en el diagrama.
- Modo conectado: lo que se demuestra corre sobre la API y SQL Server; lo que sigue simulado se dice con claridad.
- Compromiso: pantallas definitivas en Android Studio (XML y Kotlin) por el equipo, con autor en cada commit. Gustavo: reserva, confirmación y predicción; Hugo: inicio de sesión, pago, comprobante y notificaciones; Jonathan: administración y servicios.

Presupuesto y cronograma



- Catorce actividades del 22 de octubre al 28 de noviembre de 2026, seis horas semanales por integrante (108 horas).
- Valor de referencia 16.890.500 COP con 10 % de imprevistos; desembolso real 66.000 COP: herramientas gratuitas y equipos propios.
- Siguientes: interfaces nativas y conexión completa (act. 3 y 4), reentrenamiento con historial real (7), pruebas de RNF con evidencias (8 a 10), APK firmada y sustentación 27 y 28 de noviembre.

La aplicación en vivo



- Es la misma aplicación de la APK, en modo demostración (datos en memoria): se puede usar desde aquí con el puntero.
- Recepcionista: recepcion1@hotel.com · Hotel2026*. Cliente: laura@correo.com · Cliente2026*. Administrador: admin@hotel.com.
- Flujo sugerido: Reservar → tocar un tipo → elegir habitación → documento 2020202020 → Confirmar o Apartar sin pago → Historial.
- El botón Ver servidor muestra las llamadas a la API y las instrucciones SQL que ejecuta la transacción.
- En la auditoría la demostración real se hace en el teléfono con el modo conectado y la base de datos abierta en SSMS.

CIERRE

De una propuesta a una aplicación que guarda sus datos

Interfaz móvil, API con la lógica de negocio y base de datos con la transacción que evita la sobreventa. Lo pendiente está en el cronograma.

Gracias · Preguntas